AgentCART / ShopMind Implementation Plan

Conversational In-App Checkout • Agentic Commerce • Razorpay Test Mode

A phased, task-level engineering plan for building the platform described in the AgentCART Full Detailed Project Description & Technical Specification.

Item	Summary
Total duration	9 phases across ~9-10 weeks (hackathon-paced); extend 1.5-2x for a production-grade rollout
Core stack	React + Vite + Tailwind (frontend) · FastAPI + Python (backend) · PostgreSQL · LLM tool-calling agent · Razorpay Test Mode
Delivery style	Two-track build: catalogue/commerce backbone in parallel with the AI agent + policy engine, converging at the confirmation gate
Guiding principle	The AI can act, but only through backend tools; price, stock, policy, and payment truth always live server-side

1. Plan Objectives

This plan translates the ShopMind / AgentCART specification into a build sequence a small team can execute. It breaks the project into nine phases, each with concrete tasks, deliverables, dependencies, and acceptance criteria, so progress can be checked against working software rather than just documentation. The sequencing keeps the two hardest risk areas — agent tool-calling correctness and payment-state reconciliation — on the critical path early enough to leave room for hardening.

- Ship a working catalogue and cart backbone before layering the AI agent on top of it.
- Keep the AI strictly inside a controlled tool layer; never let it compute price, stock, or payment truth.
- Treat the checkout confirmation gate and Razorpay webhook reconciliation as the two must-get-right subsystems.
- Build mock-mode support early so frontend work is never blocked on backend readiness.
- Close with reliability (idempotency, retries, audit) and evaluation before deployment, not after.

2. Phase Timeline (Suggested)

#	Phase	Suggested Timing	Primary Outcome
1	Foundation	Days 1-2	Project scaffolding, schema, seed data
2	Catalogue	Days 2-4	Browsable, searchable product experience
3	Cart	Days 3-5	Authoritative shared cart, backend totals
4	AI Agent	Days 5-7	Conversational assistant with real tools
5	Policy Layer	Days 6-8	Deterministic discount/inventory/confirmation gates
6	Razorpay	Days 7-9	Test-mode order, Checkout, webhook verification
7	Reliability	Days 9-10	Retries, idempotency, audit trail
8	Evaluation	Days 10-11	Scenario tests, guardrail tests, metrics
9	Deployment	Days 11-12	Hosting, env config, docs, demo polish

Phases 3-4 and 5-6 are designed to overlap: cart work unblocks agent cart-tools before the catalogue phase is fully polished, and the policy layer can be built against a stub cart while Razorpay integration starts on the confirmation contract.

Phase 1: Foundation

Goal: Stand up the repo, tooling, and data layer so every later phase has a stable base to build on.

Tasks

- Initialize monorepo (frontend/, backend/, scripts/, docs/) per the repository structure.
- Scaffold React + Vite + JavaScript app; configure Tailwind CSS and React Router.
- Set up Zustand stores (session, cart, chat, payment) as empty shells.
- Scaffold FastAPI app with app/api, app/agent, app/policy, app/services, app/db packages.
- Configure PostgreSQL and SQLAlchemy session management; write initial Alembic/SQL migrations for the full schema (users, products, merchant_policies, carts, cart_items, conversations, messages, orders, order_items, payments, audit_logs).
- Write scripts/seed_products.py to load a realistic seed catalogue (varied categories, prices, stock, ratings).
- Implement authentication (JWT/session) endpoints: register, login.
- Set up the Axios/Fetch API service layer (api.js) with base URL and auth header injection.
- Add .env.example files for both frontend and backend; wire VITE_USE_MOCK_API for mock mode.

Deliverables

- Running frontend shell with routing stubs for all pages in the routes table.
- Running FastAPI backend with health check endpoint (GET /api/v1/health).
- PostgreSQL schema created and seeded with sample products.
- Working register/login flow returning a valid token.

Dependencies

None — this is the starting phase.

Acceptance Criteria

- `docker-compose up` (or equivalent) brings up frontend, backend, and database together.
- A new user can register and log in end-to-end.
- Seed script populates at least 20-30 products across multiple categories.

Phase 2: Catalogue

Goal: Deliver the traditional shopping entry point: a searchable, filterable, responsive product catalogue.

Tasks

- Build GET /api/v1/products with query, category, price range, rating, in_stock, sort, and pagination.
- Build GET /api/v1/products/{product_id} for product details and live availability.
- Implement product_service for search/filter/sort logic server-side.
- Build Catalogue.jsx: product grid, search box, category nav, filter panel, sort control.
- Build ProductDetails.jsx: gallery, description, price, rating, stock, quantity selector, Add to Cart.
- Add 'Ask ShopMind' entry points from both the catalogue and product detail pages, carrying product context.
- Implement responsive layout and loading/error/empty states for catalogue and product views.

Deliverables

- Fully functional catalogue with search, category filters, price/rating/discount filters, and sorting.
- Product details page with add-to-cart and AI hand-off.
- productService.js consumed by components with no raw API URLs in components.

Dependencies

Phase 1 (schema, seed data, API scaffolding, routing).

Acceptance Criteria

- Searching, filtering, and sorting return correct, paginated results from the backend (not client-only filtering).
- Add-to-cart from the catalogue works without navigating away from the grid.
- Product details page reflects live stock and price from the database.

Phase 3: Cart

Goal: Build the single authoritative cart that both the catalogue UI and the AI agent read and write.

Tasks

- Implement cart_service: create/get active cart, add item, update quantity, remove item, recalculate totals.
- Build POST /api/v1/cart, GET /api/v1/cart/{cart_id}, item add/update/delete endpoints, and GET /api/v1/cart/{cart_id}/summary.
- Enforce server-side validation of product existence and stock on every mutation.
- Add a cart `version` field to support stale-update detection.
- Build Cart.jsx and cartStore.js; ensure cart state is shared/refetched consistently between catalogue and assistant views.
- Ensure one active cart per authenticated user/session, referenced by the same cart_id from both entry points.

Deliverables

- Cart REST API with authoritative subtotal/discount/total calculation.
- Cart page showing line items, quantity controls, and remove actions.
- Shared cart state visible identically from catalogue and (soon) the AI assistant.

Dependencies

Phase 1 (schema) and Phase 2 (product data/service) for stock and price lookups.

Acceptance Criteria

- Adding the same product from the catalogue and later from a script/tool call updates one and the same cart.
- Totals are always computed server-side; the frontend never derives totals on its own.
- Attempting to add an out-of-stock item is rejected with a clear error.

Phase 4: AI Agent

Goal: Introduce the conversational shopping assistant, restricted to a controlled backend tool layer.

Tasks

- Define Pydantic tool schemas for search_products, get_product_details, add_to_cart, remove_from_cart, get_cart, and calculate_total (per the AI Agent Tool Specification).
- Write the system prompt / agent rules: ask for clarification when ambiguous; never invent products, prices, stock, or discounts; never claim payment success without verified state; explain rejected actions in plain language.
- Implement agent.py orchestration using the LLM API's native tool/function calling.
- Build POST /api/v1/chat and GET /api/v1/conversations/{id}; persist messages and tool_call_id linkage.
- Build Assistant.jsx chat UI and chatStore.js; render product recommendations and cart updates from structured agent output (never raw tool call payloads).
- Wire 'Ask ShopMind' product-context hand-off from Phase 2 into a pre-seeded conversation turn.

Deliverables

- Conversational assistant that can discover products, explain recommendations, and modify the shared cart.
- Tool-calling layer that only exposes read-only search/detail tools and validated cart-mutation tools at this stage.
- Conversation history persisted and retrievable.

Dependencies

Phase 2 (product search/service) and Phase 3 (cart service) as the tools the agent calls.

Acceptance Criteria

- The agent never fabricates a product, price, or stock figure not returned by a tool call.
- A product added via chat immediately appears in the Cart page and vice versa.
- Raw tool-call JSON is never rendered to the customer; only natural-language + structured product cards are.

Phase 5: Policy Layer

Goal: Add the deterministic guardrails that keep the AI from controlling price, discounts, or purchase authorization.

Tasks

- Implement policy/engine.py and policy/rules.py: validate_discount, validate_inventory, validate_cart_owner, calculate_authoritative_total, validate_confirmation, validate_order_state, validate_payment_amount, check_duplicate_payment.
- Model merchant_policies (max discount %, max discount amount, minimum cart value, coupon rules, validity, eligibility) as configuration, not hard-coded logic.
- Wire apply_merchant_discount tool through the policy engine; add POST /api/v1/cart/{cart_id}/discount.
- Build order_service: POST /api/v1/orders/preview (validated purchase preview) and POST /api/v1/orders/{id}/confirm (explicit confirmation state).
- Build Checkout.jsx as a true confirmation gate: exact products, quantities, discounts, and final amount, editable only by returning to cart.
- Ensure totals are recalculated immediately before payment, not reused from an earlier snapshot.

Deliverables

- Policy engine enforcing discount, inventory, ownership, and confirmation rules independent of the LLM.
- Checkout page that blocks payment-order creation until an explicit Confirm & Pay action.
- Order preview/confirm API pair with server-recorded confirmation state.

Dependencies

Phase 3 (cart) for totals, Phase 4 (agent) for the discount-request tool call.

Acceptance Criteria

- A request for a discount above merchant limits is rejected with a customer-friendly explanation, not silently adjusted.
- The confirmed order amount always matches a fresh server-side recalculation, never a client-supplied number.
- Confirmation state is persisted before any payment-order creation is attempted.

Phase 6: Razorpay Integration

Goal: Connect the confirmed order to Razorpay Test Mode and verify payment outcomes from the backend, not the frontend.

Tasks

- Implement payment_service.create_order calling Razorpay to create a Test Mode order tied to a CONFIRMED order.
- Build POST /api/v1/payments/razorpay/order and GET /api/v1/payments/{order_id}/status.
- Integrate Razorpay Checkout on the frontend using the public key and order ID only (no card/UPI form of ShopMind's own).
- Implement POST /api/v1/webhooks/razorpay: verify signature, reconcile payment event against the order, and update order/payment status.
- Model the payment state machine end-to-end: CART_READY -> AWAITING_CONFIRMATION -> ORDER_CONFIRMED -> PAYMENT_ORDER_CREATED -> CHECKOUT_OPEN -> PAYMENT_SUCCESS/PAYMENT_FAILED -> ORDER_PAID / RETRY_AVAILABLE.
- Build Orders.jsx and OrderDetails.jsx to display verified backend state, not client-assumed state.

Deliverables

- End-to-end payment flow from Confirm & Pay through Razorpay Checkout to a verified order status.
- Webhook endpoint that reconciles payment events idempotently.
- Order history and detail pages reflecting only backend-verified payment state.

Dependencies

Phase 5 (confirmed order + authoritative amount) is a hard prerequisite.

Acceptance Criteria

- The frontend never marks an order PAID on its own; it only displays what the backend reports after webhook verification.
- A simulated Test Mode payment failure results in PAYMENT_FAILED and a retry path, never a false success.
- Razorpay secret key and webhook secret exist only in backend environment variables.

Phase 7: Reliability

Goal: Harden the system against duplicate events, interrupted flows, and unavailable dependencies.

Tasks

- Add idempotency keys / state checks to payment-order creation and webhook processing (check_duplicate_payment).
- Implement a controlled retry path for failed payments that reopens Checkout without creating a duplicate charge.
- Handle out-of-stock, invalid-discount, expired/interrupted checkout, and backend-unavailable cases per the Failure Handling matrix.
- Implement audit_service and the full audit_logs event set (USER_MESSAGE, AGENT_DECISION, TOOL_CALL, TOOL_RESULT, POLICY_CHECK, CART_UPDATE, DISCOUNT_APPLIED/REJECTED, FINAL_TOTAL, USER_CONFIRMATION, PAYMENT_ORDER_CREATED, CHECKOUT_OPENED, WEBHOOK_RECEIVED/VERIFIED, PAYMENT_SUCCESS/FAILED, ORDER_PAID/FAILED, RETRY_REQUESTED).
- Build GET /api/v1/audit/{session_id} and a minimal internal view for debugging demo runs.
- Add authorization checks so one user cannot read or mutate another user's cart or order.

Deliverables

- Idempotent payment and webhook handling verified with replayed events.
- Complete audit trail queryable per session/order.
- Documented failure-mode behavior matching the specification's failure table.

Dependencies

Phase 6 (payment flow) must exist before reliability work has something to harden.

Acceptance Criteria

- Replaying the same webhook event twice does not double-charge, duplicate the order, or corrupt state.
- Every state transition in the payment state machine writes a corresponding audit event.
- A user cannot fetch or modify another user's cart/order via direct API calls.

Phase 8: Evaluation

Goal: Prove the system behaves correctly under the demo scenarios and guardrail tests before deployment.

Tasks

- Write Pytest suites for product_service, cart_service, policy engine, order_service, and payment_service.
- Write agent evaluation tests: correct tool selection and valid arguments for representative shopping requests.
- Measure task completion rate against a fixed set of predefined shopping requests.
- Write guardrail tests: invalid discount requests, unauthorized payment-order creation attempts, ambiguous requests requiring clarification.
- Run the seven recommended demo scenarios end-to-end (normal purchase, AI shopping, merchant discount, invalid discount, payment failure, product-context hand-off, duplicate webhook event) and fix any gaps found.
- Add basic frontend tests for critical flows (add-to-cart, checkout gate, order confirmation display).

Deliverables

- Automated backend test suite covering services, policy engine, and payment reconciliation.
- Recorded results for all nine metrics in the Testing and Evaluation table (catalogue, AI tool use, shopping task, guardrails, payment gate, payment reliability, duplicate prevention, audit completeness, UX).
- All seven demo scenarios passing end-to-end.

Dependencies

Requires all prior phases functionally complete.

Acceptance Criteria

- No demo scenario requires manual workaround or hidden setup to succeed.
- Guardrail tests demonstrate the policy engine, not the LLM, is what blocks invalid actions.
- Test suite runs in CI (or locally) and passes cleanly before deployment.

Phase 9: Deployment

Goal: Ship the frontend and backend, finalize configuration, and prepare demo-ready documentation.

Tasks

- Deploy frontend to Vercel (or equivalent static/SPA host) with production environment variables.
- Deploy backend to Render/Railway (or equivalent) with PostgreSQL, LLM API key, and Razorpay Test Mode credentials configured as secrets.
- Register the production/staging webhook URL with Razorpay Test Mode and verify signature validation against the live endpoint.
- Write docs/architecture.md, docs/api.md, and docs/demo.md summarizing system design, endpoints, and how to run the demo scenarios.
- Finalize README.md with setup instructions, environment variable reference, and mock-mode instructions.
- Polish UX rough edges surfaced during evaluation (loading states, error messages, empty states, accessibility basics).

Deliverables

- Publicly reachable frontend and backend with working end-to-end checkout in Razorpay Test Mode.
- Verified webhook integration against the deployed backend URL.
- Complete documentation set (architecture, API, demo script) and a polished README.

Dependencies

Requires Phase 8 evaluation to have passed; ideally no open guardrail failures at deployment time.

Acceptance Criteria

- A fresh reviewer can clone the repo, follow the README, and run the demo scenarios without additional context.
- No secrets (Razorpay keys, LLM API key, JWT secret) are present in frontend bundles or committed to the repository.
- The deployed webhook endpoint successfully reconciles a live Test Mode payment event.

3. Standing Guardrail (Applies to Every Phase)

Throughout all nine phases, the following must remain outside the LLM's control and be enforced purely by backend code and the policy engine. Any implementation task that would give the model direct authority over one of these should be redesigned before it ships.

Control	Where it actually lives
Final price	Computed by cart_service / policy engine from live product and discount data
Stock truth	Read from PostgreSQL by product_service on every mutation
Merchant policy	Configured in merchant_policies table, enforced by policy engine
Payment credentials	Held only in backend environment variables
Payment success state	Determined solely by verified Razorpay webhook reconciliation
User authorization	Enforced by auth/session checks on every cart/order endpoint
Webhook verification	Signature-checked server-side before any state change
Database access	Backend services only; the agent never receives direct DB or credential access

4. Suggested Team Roles

Role	Primary Responsibilities
Frontend engineer	Catalogue, product details, cart, checkout, orders UI; API service layer; Zustand stores
Backend engineer	FastAPI services, database schema, cart/order/payment endpoints, webhook handling
Agent/AI engineer	Tool schemas, prompts, agent orchestration, conversation persistence, agent evaluation
Full-stack / integration	Policy engine, Razorpay integration, reliability/idempotency, deployment
QA / evaluation owner	Demo scenarios, guardrail tests, metrics tracking, audit-trail verification

On a smaller team, these roles can be merged; the important boundary to preserve is that the person building the AI agent is not the sole reviewer of the policy engine that constrains it.

5. Key Risks and Mitigations

Risk	Mitigation
Agent invents product/price/stock data	Tool-only architecture; agent evaluation tests in Phase 8; explicit prompt rule against invention
Duplicate payment or double-charge	Idempotency keys, check_duplicate_payment, webhook replay tests in Phase 7
Frontend blocked waiting on backend	Mock mode (VITE_USE_MOCK_API) built in Phase 1, same interface as real services
Discount abuse / policy bypass	Discounts always routed through policy engine; guardrail tests in Phase 8

Risk	Mitigation
Payment state shown before verified	Frontend only displays backend-confirmed state; no optimistic 'success' UI
Secrets leaking to frontend	Environment separation enforced in Phase 1 and checked again at deployment (Phase 9)

6. Milestone Checkpoints

Milestone	Checkpoint	Definition of Done
M1	End of Phase 2	Customer can browse, search, and view products end-to-end
M2	End of Phase 4	Customer can complete a full AI-assisted shopping conversation that updates a real cart
M3	End of Phase 6	A confirmed order can be paid through Razorpay Test Mode and verified via webhook
M4	End of Phase 8	All seven demo scenarios pass and guardrail/reliability tests are green
M5	End of Phase 9	Publicly deployed, documented, demo-ready system

---

AgentCART / ShopMind — Implementation Plan derived from the Full Detailed Project Description & Technical Specification.